

AI-Assisted Exploration of a 4KB SRAM Design Flow Using OpenRAM and SKY130

Sanchaika Singh

VSD Internship Program | Week1 Task Report

Abstract: This Week 1 study examines the fundamentals of SRAM design and how a 1024x32 (4KB) SRAM macro can be approached using open-source tools and AI-assisted workflows. The reference repository targets a 1.8 V SKY130 SRAM with access time below 2.5 ns using OpenRAM. The current goal is not full layout reproduction, but understanding the architecture, required inputs, expected outputs, setup risks, and debugging strategy.

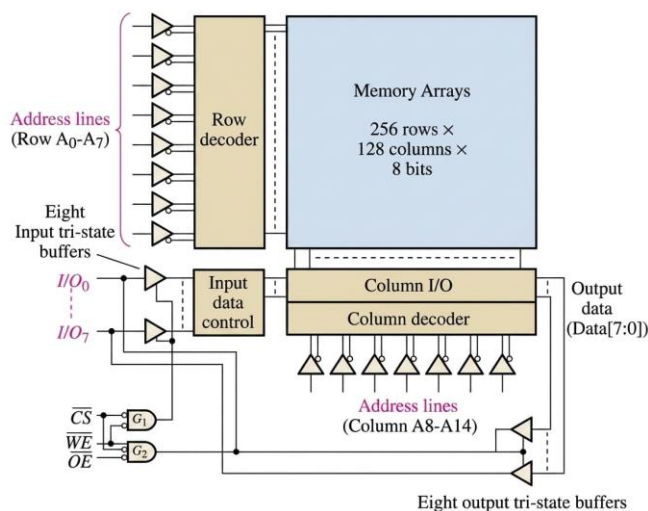
I. Introduction

SRAM is a volatile memory technology that retains stored information as long as power is supplied. Due to its fast access time and simple read operation, SRAM is extensively used in processor cache memories and high-performance computing systems.

The objective of this preliminary research is to understand how OpenRAM converts a memory specification into a manufacturable SRAM macro using the SKY130 technology platform.

II. SRAM Architecture

A SRAM macro is built from repeated 6T bitcells, wordline drivers, bitlines, precharge circuits, sense amplifiers, write drivers, row/column decoders, control logic, and input/output registers. A 6T SRAM cell uses two cross-coupled CMOS inverters to store data and two access NMOS transistors controlled by the wordline.



III. OpenRAM and SKY130 Toolchain

OpenRAM is an open-source memory compiler that generates SRAM macros from user-defined specifications such as word size, memory depth, ports, and operating conditions. It automatically creates the SRAM array, peripheral circuits, and associated design files.

SKY130 is an open-source 130 nm PDK that provides transistor models, design rules, and verification data required for physical implementation. Together, OpenRAM and

SKY130 enable the generation of GDS, LEF, SPICE, Liberty, and Verilog files for ASIC memory design.

IV. SRAM Building Blocks and Design Views

SRAM generation begins with specification such as size, word width, voltage, process corner, timing target.

For this project, the target is **1024x32 SRAM, 32 Kbits / 4 KB, using SKY130 at 1.8 V.**

OpenRAM builds the SRAM macro using pre-designed blocks such as the **6T bitcell, sense amplifier, write driver, precharge circuit, and decoder**. These blocks are provided in multiple views, including **GDS, SPICE, LEF, and Liberty (.lib)**, enabling layout generation, simulation, timing analysis, and physical design integration within the SKY130 technology.

Inputs: SRAM specifications (word size, number of words, supply voltage, and process corner) along with SKY130 technology files.

Outputs: SRAM layout, netlists, timing models, and physical design abstractions.

V. Common Failure Points and Debugging

Setup/path issues: Missing environment variables, wrong SKY130 PDK paths, or incorrect SPICE model paths can stop OpenRAM, NGSPICE, or layout tools from running correctly.

Technology/layout mapping issues: Mismatched layer names, missing boundary layers, unsupported SKY130 contacts, and incorrect FEOL stack mapping can cause contact placement, routing, DRC, or LVS errors.

Debug strategy: Verify tools and paths first, run small 1-bit/custom-cell simulations, validate GDS/SPICE/LEF files, then test a small SRAM before attempting the full 1024x32 macro.

VI. Key Observation

From this Week 1 study, I understood that SRAM design is not just about running OpenRAM and getting an output file. It requires understanding how the memory cell works, how read and write operations happen, and how blocks like precharge, sense amplifier, and write driver support the main array. I also observed that the tool flow depends heavily on correct SKY130 PDK setup, layer mapping, SPICE models, and custom cell files. AI tools are useful for breaking the repo into smaller tasks and explaining errors. "For instance, ChatGPT was prompted to 'Convert the SRAM_SKY130 repository into step-by-step OpenRAM setup commands,' which helped isolate the missing environment variables discussed in Section V." but the final understanding must come from checking each step carefully.